

操作系统基础与程序运行环境

从裸硬件到受保护、可共享、可管理的执行环境

408 操作系统专题手册 OS-00

母问题：操作系统为什么存在？

一段普通程序为什么可以像“独占一台机器”一样运行，
而真实硬件明明有限、共享、异步，并且可能发生故障？

本册不从“操作系统有哪些功能”开始背诵，而从裸硬件不能直接满足程序需求开始推导：

$$\text{Finite Physical Resources} \xrightarrow[\text{Protection + Policy}]{\text{Abstraction + Virtualization}} \text{Program-visible Managed Environment}$$

全书继续沿用 OS 统一推演引擎：

$$\text{State} \rightarrow \text{Transition} \rightarrow \text{Failure} \rightarrow \text{Property} \rightarrow \text{Mechanism} \rightarrow \text{Tradeoff}$$

但 OS-00 的任务不是深入某个局部机制，而是建立后续所有 Topic 共用的坐标系、运行入口和边界。

Owner / Stop Boundary

本册完整拥有：OS 基本概念、发展逻辑、408 四特征、程序链接与装入的入口语义、运行时映像的基础区分、OS 结构、系统引导、机器级虚拟化，以及这些概念与后续 Topic 的导航关系。

以下内容只给**最小接口**，不重新拥有内部机制：

- user/kernel mode、system call、exception、interrupt 的控制权细节 → OS-01/02；
- 同步互斥、PV、管程、死锁 → OS-03；
- 地址翻译、页表、缺页、页置换、COW/mmap → OS-04；
- driver、DMA、I/O request、buffering、device scheduling → OS-05；
- pathname、fd/OFD、inode、文件分配与一致性 → OS-06/07；
- 特权级、MMU、IRQ controller 的硬件实现 → CPU × OS Bridge。

目录

1 第一章：先看裸机——程序真正需要的不是“硬件”，而是可用的机器	2
1.1 从一条指令开始：裸机模型看起来很简单	2
1.2 Abstraction：先把“怎么做”隐藏成“我能做什么”	2
1.3 Virtualization：把有限资源变成多个可管理的逻辑实例	3

1.4	Protection / Isolation: 虚拟化必须带权限边界	3
1.5	Resource Management: 虚拟化之后还必须决定“谁先用、用多少”	4
1.6	OS 的功能与服务: 不要再背“处理机/存储器/文件/设备管理”四张孤立卡片	4
2	第二章: OSTEP 三条主线与 408 四特征——两套坐标, 不要硬凑	5
2.1	OSTEP 的 Three Easy Pieces 是三类“系统问题”	5
2.2	408 四特征描述的是 OS/多道环境呈现出来的性质	5
2.3	四特征的因果关系比并列背诵更重要	5
3	第三章: 操作系统为什么一步步变成今天这样——用失败推动历史	6
3.1	历史不是年代记, 而是“旧运行方式的瓶颈”	6
3.2	人工操作: 机器时间被人工交接浪费	6
3.3	批处理与单道: 减少人工切换, 但 I/O 一来 CPU 仍空闲	6
3.4	多道程序设计: 用并发覆盖等待	6
3.5	分时系统: 吞吐率不够, 人还需要“快响应”	7
3.6	实时系统: 平均快还不够, 必须满足时限约束	7
4	第四章: 程序运行环境——Program、Executable、Process Image 与 Process 不是同一对象	7
4.1	最容易缺的一条主链: 静态描述怎样变成运行实体	7
4.2	链接到底在解决什么? ——多个模块怎样成为一个可执行整体	8
4.3	静态链接与动态链接: 绑定发生在什么时候?	8
4.4	三种经典装入方式: 真正变化的是“地址在什么时候确定”	8
4.5	装入不是“把整个程序文件一次性复制进物理内存”	9
4.6	运行时内存映像: 知道有哪些区域, 但不要在这里讲页表	9
5	第五章: 程序怎样跨过权限边界——本册只建立运行环境入口	10
5.1	User Mode 与 Kernel Mode: 为什么必须有权限层级	10
5.2	为什么需要系统调用?	10
5.3	System Call、Exception、Interrupt 只在此处建立分类接口	10
6	第六章: 操作系统结构——真正的问题是“哪些能力必须放在特权核心里?”	10
6.1	不要背优缺点表, 先建立比较轴	10
6.2	宏内核 (Monolithic Kernel): 大量核心服务共处内核地址空间	11
6.3	模块化: 解决“一个大内核怎样组织和扩展”	11
6.4	分层结构: 限制跨层依赖	12
6.5	微内核 (Microkernel): 把高层服务移出特权核心	12

6.6	外核 (Exokernel): 更进一步, 把“保护”和“资源管理策略”分开	12
7	第七章: 系统引导——OS 自己也必须先被加载和获得控制权	12
7.1	Bootstrap Paradox: 还没有 OS 时, 谁来加载 OS?	12
7.2	Firmware: 先把平台带到可加载下一阶段的状态	13
7.3	Bootloader: 找到内核、准备参数、移交控制	13
7.4	Kernel 启动之后还没有“完整用户环境”	13
8	第八章: 虚拟机——OS 虚拟化资源, 不等于 VMM 虚拟整台机器	13
8.1	两种 virtualization 必须分层	13
8.2	VMM 的母问题: 多个 OS 怎样安全共享同一台物理机器?	14
8.3	Type 1 / Type 2 只作为部署结构理解	14
9	第九章: 一张总图把 OS-00 挂到后续所有 Topic	14
10	第十章: OS-00 做题控制器——先定位层, 再调用 Owner	15
10.1	七问协议	15
10.2	高频概念区分清单	15
10.3	忘掉名词时怎样重新推出来?	16
	参考与工程边界来源	16

1 第一章：先看裸机——程序真正需要的不是“硬件”，而是可用的机器

1.1 从一条指令开始：裸机模型看起来很简单

若只从 CPU 的视角观察，一个正在运行的程序似乎只是不断重复：

Fetch → Decode → Execute

程序读写内存、执行算术、分支和函数调用，直到结束。

问题在于：这只描述了单个程序在理想机器上的局部执行，不描述真实系统怎样让很多程序安全、方便、有效地共享同一套硬件。

把真实机器放回来，立刻出现四组矛盾：

1. **资源有限**：CPU 核心、内存、设备和持久存储都有限；
2. **多个程序竞争**：多个程序同时想用 CPU、RAM 和设备；
3. **直接访问危险**：任何程序若可任意修改页表、设备控制寄存器或别人的内存，隔离立即失效；
4. **硬件接口复杂且异步**：设备速度不同，完成事件可能在程序执行的任意时刻到来。

因此程序真正希望得到的不是“某块裸硬件”，而是一台具有稳定接口和受保护语义的**逻辑机器**。

第一条生成链：操作系统的双视角 (Twin Viewpoint)

OS = Resource Manager + Extended Machine / Interface

分析操作系统的任何功能时，固定从双重视角看：

1. **自下而上：资源管理者 (Resource Manager)**：面对物理硬件（CPU、内存、设备、磁盘），负责分配、复用、隔离与并发管控；
2. **自上而下：扩展机与抽象层 (Extended Machine / Interface)**：面向应用程序，隐藏底层接口细节，把粗糙的物理硬件重新解释为优雅的逻辑抽象（进程、虚拟内存、文件、系统调用）。

1.2 Abstraction：先把“怎么做”隐藏成“我能做什么”

硬盘真正暴露的是块、扇区、控制命令；应用想要的是 `/home/a.txt`。物理内存是一串字节位置；程序想要的是稳定的地址空间。CPU 是有限执行单元；程序想要的是“我的执行流可以继续运行”。

因此第一层不是虚拟化，而是**抽象 (Abstraction)**：

Complex Physical Interface → Stable Logical Interface

典型例子：

物理资源	程序希望看到的抽象	后续 Owner
CPU	process / thread、可运行执行流	OS-01/02
内存	address space、可保护的虚拟地址	OS-04
存储设备	file / directory / namespace	OS-06/07
I/O 设备	统一或受控的 I/O interface	OS-05

Abstraction ≠ Virtualization

抽象回答：“程序应该通过什么对象和接口使用资源？”

虚拟化回答：“怎样让有限物理资源表现得像更丰富、可独立使用的逻辑资源？”

两者经常一起出现，但不能当作同义词。

1.3 Virtualization：把有限资源变成多个可管理的逻辑实例

OSTEP 将 virtualization 作为 OS 的核心组织原则之一：OS 接收 CPU、memory、disk 等物理资源，把它们转化成更通用、更方便使用的虚拟形式，同时通过 mechanism 与 policy 决定怎样共享这些资源。

最直观的例子是 CPU：只有少量 CPU 核心，却能让大量程序都表现为“正在运行”。这不是凭空创造算力，而是：

Physical CPU Time $\xrightarrow{\text{Multiplexing + State Preservation}}$ Many Logical Execution Streams

内存同理：

Physical Memory $\xrightarrow{\text{Mapping + Protection}}$ Per-process Address Spaces

题目信号

看到“多个进程都认为自己从同一虚拟地址开始”“单核却有多个程序同时运行”“逻辑资源数量大于物理资源数量”时，第一动作不是背算法，而是问：

物理资源是什么？虚拟对象是什么？谁负责映射和复用？

1.4 Protection / Isolation：虚拟化必须带权限边界

如果只是“共享”，没有隔离，一个程序就能破坏另一个程序或内核。因此：

Sharing without Protection $\Rightarrow$ Unsafe System

OS 必须控制：

- 哪些指令普通程序不能直接执行；
- 哪些地址可以访问；

- 哪些资源对象具有访问权限；
- 哪些状态只能由内核修改。

Protection 是后续 user/kernel mode、memory protection、file permission、capability 等机制共享的上位问题。本册只建立这个目标，不在这里展开硬件特权级和具体权限实现。

1.5 Resource Management：虚拟化之后还必须决定 “谁先用、用多少”

当多个程序竞争同一资源时，仅有 mechanism 不够，还需要 policy：

Mechanism: 能不能做? Policy: 选择谁做?

例如：

- CPU 可以被切换，这是 mechanism；下一位运行谁，是 scheduling policy；
- 页面可以被换出，这是 mechanism；淘汰哪一页，是 replacement policy；
- 磁盘请求可以排队，这是 mechanism；下一请求选谁，是 device scheduling policy。

这解释了 OS 为什么也被称为 **resource manager**。

1.6 OS 的功能与服务：不要再背 “处理机/存储器/文件/设备管理” 四张孤立卡片

传统教材常把 OS 功能列成处理机管理、存储器管理、文件管理、设备管理，再补用户接口。更好的挂法是按**资源域 + 对外服务**理解：

资源/对象域	OS 必须解决的母问题	后续 Owner
CPU / execution	谁能运行、何时运行、怎样保存/恢复执行上下文	OS-01/02
shared state	多个执行流交错时怎样保持正确性与进展	OS-03
memory	地址空间、分配、保护、驻留和回收怎样管理	OS-04
I/O device	请求怎样提交、等待、完成并与设备速度差异解耦	OS-05
persistent storage	名字、对象、内容、空间和崩溃后一致性怎样维持	OS-06/07

从应用视角，OS 还必须提供**可调用接口**：程序执行、I/O、文件/目录操作、通信、错误处理、资源分配、保护等最终通过 API / system-call boundary 暴露。这里要保持：

Library API ≠ System Call ≠ Kernel Internal Function

一个库函数可能完全在用户态完成，也可能封装一个或多个 system calls；system call 是受控进入内核服务的接口语义。

功能题第一动作

题目问“操作系统功能/服务”时，先问它在管理哪个资源域、向谁提供什么可观察接口，再把它送到后续 Owner。不要把“管理功能”和“用户接口”混成同一层。

2 第二章：OSTEP 三条主线与 408 四特征——两套坐标，不要硬凑

2.1 OSTEP 的 Three Easy Pieces 是三类“系统问题”

OSTEP 用三条主线组织操作系统：

Virtualization + Concurrency + Persistence

它们分别追问：

- 1. **Virtualization**：怎样把 CPU、memory 等物理资源变成可共享的逻辑资源？
- 2. **Concurrency**：多个活动同时推进时，交错和共享状态怎样保持正确？
- 3. **Persistence**：数据怎样跨越进程生命周期和断电继续存在？

它们是课程组织轴 / 问题轴，不是 408 四特征的英文翻译。

2.2 408 四特征描述的是 OS/多道环境呈现出来的性质

特征	一句话抓手	真正需要区分的点
并发	多个事件在一段时间内交替推进	并发 ≠ 并行；单核也可并发
共享	多个执行实体共同使用有限资源	互斥共享与同时共享不同
虚拟	物理实体被变换成逻辑上的多个/更易用实体	虚拟 ≠ “不存在”
异步	进程推进速度和完成时刻具有不可预知性	异步来自调度、竞争、设备完成等不确定交错

2.3 四特征的因果关系比并列背诵更重要

不要把四个词当四张卡片。更好的生成关系是：

有限资源被共享 ⇒ 多个活动并发推进 ⇒ 执行次序和进度呈异步性

同时，为了让共享更方便、安全，OS 使用虚拟化：

Physical Sharing $\xrightarrow{\text{Virtualization + Protection}}$ Logical Isolation / Managed Sharing

408 高频辨析

- **并发 $\neq$ 并行**：并发强调时间段内多个活动都在推进；并行强调同一时刻真正同时执行。
- **共享是并发的资源条件之一**：若完全没有资源关系，很多并发冲突就不会出现。
- **异步不是“使用异步 I/O”** 它是更一般的推进次序不确定性。
- **虚拟不是模拟器专属术语**：CPU 时间复用、地址空间、文件抽象都体现广义虚拟化思想。

3 第三章：操作系统为什么一步步变成今天这样——用失败推动历史

3.1 历史不是年代记，而是“旧运行方式的瓶颈”

OS 的发展可以用两个不断变化的目标驱动：

Resource Utilization

Response / Interaction / Timing

3.2 人工操作：机器时间被人工交接浪费

早期模式近似：

Human Setup $\rightarrow$ Run One Job $\rightarrow$ Human Collect $\rightarrow$ Next Job

失败点：CPU 很昂贵，但大量时间在等待人工装入、切换和准备。

由此推动**自动作业接续**。

3.3 批处理与单道：减少人工切换，但 I/O 一来 CPU 仍空闲

单道批处理解决“作业自动连续执行”，但若一个作业等待慢速 I/O：

Job waits for I/O $\Rightarrow$ CPU idle

因此下一步自然不是“换一种名字”，而是：**内存里同时保留多个作业，当前作业等待时让另一个运行。**

3.4 多道程序设计：用并发覆盖等待

One Job Waits $\Rightarrow$ Run Another Ready Job

这直接带来现代 OS 的一系列问题：

- 多个作业如何共存于内存？
- CPU 下一步给谁？
- 一个程序如何不能破坏另一个程序？
- 多个程序共享设备时如何排队？

换句话说，多道程序设计不是孤立历史名词，而是进程、调度、内存保护、资源管理问题集体出现的原因之一。

3.5 分时系统：吞吐率不够，人还需要“快响应”

批处理更关心单位时间完成多少作业；交互用户更关心：

我发出请求后多久看到反馈？

于是 CPU 被更细粒度地在多个交互任务之间切换，目标从纯 utilization/throughput 扩展到 response time 与 fairness。

3.6 实时系统：平均快还不够，必须满足时限约束

实时系统的关键不是“运算速度特别快”，而是：

Correct Result + Timing Constraint

因此平均响应时间可能不是主要指标；最坏情况、deadline、可预测性变得重要。

历史题统一做法

题目问某种 OS 形态时，不先背定义，先问：

1. 上一代系统浪费在哪里？
2. 新系统主要优化 utilization、throughput、response 还是 deadline？
3. 为了这个目标，引入了什么新的共享/调度关系？

4 第四章：程序运行环境——Program、Executable、Process Image 与 Process 不是同一对象

4.1 最容易缺的一条主链：静态描述怎样变成运行实体

一段源代码离 CPU 执行还有多层变换：

Source → Object File → Executable / Shared Object → Process Image → Running Process

箭头含义分别是：编译/汇编产生目标文件；链接组合目标文件和符号关系；装入根据可执行文件描述建立运行时映像；随后 OS 建立可管理的执行实体并把控制交给程序入口。

四个对象不能混

- **Program / Executable**：静态文件；
- **Process Image**：某次执行所需的运行时地址空间内容与初始布局；
- **Process**：OS 管理的动态执行实体；
- **Address Space**：该执行上下文看到的地址集合及其映射语义，完整机制由 OS-04 拥有。

4.2 链接到底在解决什么？——多个模块怎样成为一个可执行整体

编译单元彼此独立时会留下外部符号引用。例如一个目标文件调用另一个目标文件中的函数。链接器必须解决两类核心问题：

Symbol Resolution + Relocation

- Symbol Resolution**：这个名字最终指向哪个定义？
- Relocation**：组合后对象和代码位置改变，相关引用怎样调整到正确位置？

本册只保留链接层面的语义。若题目追问逻辑地址怎样在运行期映射到物理地址，立即切到 OS-04。

4.3 静态链接与动态链接：绑定发生在什么时候？

可用一个统一问题理解：

Dependency is bound when?

方式	绑定时机	主要权衡
静态链接	生成可执行文件时	运行依赖少，但可执行文件更大、共享库代码复用差
装入时动态链接	程序开始建立运行映像时	多程序可共享库，启动时需完成依赖处理
运行时动态链接	程序运行过程中按需装入/绑定	更灵活，但运行时路径和失败情况更复杂

ELF 的动态链接模型进一步体现了这一点：可执行文件可指定 program interpreter，由动态链接器参与把 executable 和 shared objects 组合成完整 process image，再把控制交给应用程序。

4.4 三种经典装入方式：真正变化的是“地址在什么时候确定”

408 传统模型中的装入方式也不要孤立背。统一问：

程序地址在什么时候被绑定到最终运行位置？

装入方式	地址确定时机	机制与边界
绝对装入	编译/生成程序时就已知最终绝对地址	只适合运行位置预先确定的简单环境，灵活性最低
可重定位装入（静态重定位）	装入时一次性根据实际起始位置修改地址相关项	装入后通常难以再次移动；“重定位”发生在 load time
动态运行时装入（动态重定位）	程序运行期间保留逻辑/相对地址，访问时借助硬件动态形成实际地址	支持更灵活的移动与虚拟存储；具体 MMU/页表机制归 OS-04

Loading classification $\neq$ Linking classification

“静态链接 / 装入时动态链接 / 运行时动态链接”讨论**模块之间的依赖何时绑定**；
“绝对装入 / 可重定位装入 / 动态运行时装入”讨论**程序地址何时绑定到运行位置**。
两套分类使用了相似的“静态/动态/运行时”词汇，但回答的是不同问题。

4.5 装入不是“把整个程序文件一次性复制进物理内存”

这是一个非常重要的边界。

ELF 规范把 executable/shared object 看作程序的**静态表示**，执行时由系统据此创建**动态的 process image**。现代系统可以先建立映射关系，真正物理读取某些页可以推迟到发生访问时。

因此：

Load / Map a Program $\neq$ All Pages Immediately Resident

这句话只在 OS-00 建立概念边界；**为什么可以延迟、页表如何表示、Page Fault 如何修复**全部属于 OS-04。

4.6 运行时内存映像：知道有哪些区域，但不要在这里讲页表

经典进程映像可以按语义识别为：

- code / text：程序指令；
- read-only data：只读常量；
- initialized data：已初始化全局/静态数据；
- BSS：未显式初始化的静态存储对象，文件中不必为其保存等量数据；
- heap：运行时动态增长的数据区域；
- stack：函数调用、局部状态等运行时结构；
- shared libraries / mapped regions：共享对象与其他映射区域。

考试第一动作

题目出现“程序装入、链接、动态库、运行时内存映像”时，先定位题目究竟问的是哪一层：

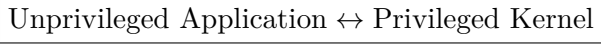
File Composition? Runtime Image? Address Translation?

前两者 OS-00；最后一个 OS-04。

5 第五章：程序怎样跨过权限边界——本册只建立运行环境入口

5.1 User Mode 与 Kernel Mode：为什么必须有权限层级

若应用程序可以直接执行所有敏感操作，Protection 目标无法成立。因此 CPU 提供不同权限状态，OS 使用它们建立：

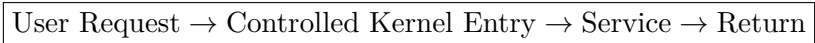


这里只需要记住：**CPU mode 是硬件执行权限状态，不等于进程状态。**

Ready/Running/Blocked 是任务调度状态；User/Kernel 是当前 CPU 权限状态。两套状态轴不能混。

5.2 为什么需要系统调用？

应用不能直接做特权操作，但又必须访问文件、设备、创建进程等内核服务，于是需要受控入口：



system call 的核心不是“函数调用名字很特殊”，而是**跨越保护边界，请求内核代为执行受保护操作。**

5.3 System Call、Exception、Interrupt 只在此处建立分类接口

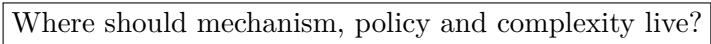
入口	与当前指令关系	核心含义
System Call	当前程序主动发起、同步	主动请求 OS 服务
Exception	由当前指令执行触发、同步	当前执行出现需要特殊处理的条件
Interrupt	通常来自外部设备、异步	外部事件要求 CPU/OS 响应

具体 trap 保存什么、硬件自动完成哪些动作、何时调度、interrupt controller 如何仲裁，全部切到 OS-01/02 与 CPU×OS Bridge。

6 第六章：操作系统结构——真正的问题是“哪些能力必须放在特权核心里？”

6.1 不要背优缺点表，先建立比较轴

所有 OS 结构问题都可以压缩成：



内核结构演进的评价轴：模块独立性

Complexity → Decomposition → Module Boundary → High Cohesion + Low Coupling → Tradeoff

操作系统结构演进的本质，是寻找**复杂度分解与可靠边界**。评价任何内核结构，固定看四组指标：

1. **High Cohesion (高内聚)**：模块内部是否专注完成单一逻辑职责（如虚拟内存管理、进程调度）；
2. **Low Coupling (低耦合)**：模块之间是否通过稳定、受限的接口通信，避免随意访问内部数据结构；
3. **Fault Isolation (故障隔离)**：一个模块（如驱动程序）崩溃，是否会拖垮整个内核；
4. **Boundary Cost (边界成本)**：跨越模块/权限边界时付出的函数调用、内核态切换或 IPC 开销。

思考任何内核架构题时，问五件事：

1. 哪些组件运行在 privileged kernel?
2. 组件之间是直接函数调用还是 IPC/message?
3. 故障隔离边界在哪里?
4. 扩展/替换组件有多容易?
5. 额外抽象或通信带来什么性能成本?

6.2 宏内核 (Monolithic Kernel)：大量核心服务共处内核地址空间

典型模型：

User Apps → Large Privileged Kernel: FS + VM + Network + Drivers + Scheduler → Hardware

优势来自**组件之间可以高效直接协作**；风险来自**大量复杂代码共享高权限和故障域**。

注意：“宏内核”不等于“所有代码写成一个源文件”。现代宏内核完全可以高度模块化。

6.3 模块化：解决“一个大内核怎样组织和扩展”

模块化强调：

Well-defined Interfaces + Replaceable / Loadable Components

它与宏内核不是互斥分类。一个系统可以是 monolithic-kernel architecture，同时支持 loadable kernel modules。

因此：

Monolithic vs Microkernel ≠ Modular vs Non-modular

6.4 分层结构：限制跨层依赖

分层思想要求高层使用低层服务：

$$L_n \rightarrow L_{n-1} \rightarrow \cdots \rightarrow L_0$$

优点是接口和责任清晰；代价是现实需求未必天然适合严格层级，额外层次也可能引入路径成本。

6.5 微内核（Microkernel）：把高层服务移出特权核心

微内核的生成问题是：

What is the minimum that must remain privileged?

典型结构：

Applications / User-space Servers $\xrightarrow{IPC}$ Minimal Kernel Mechanisms $\rightarrow$ Hardware

seL4 的官方描述非常适合作为工程边界：微内核只保留控制物理地址空间、interrupt 和 processor time 等最小机制，高层文件、进程、socket 等抽象可以由用户态系统服务构造。

于是 tradeoff 很清楚：

Smaller Privileged TCB / Better Isolation $\leftrightarrow$ More IPC / Boundary-crossing Cost

6.6 外核（Exokernel）：更进一步，把“保护”和“资源管理策略”分开

外核思想的核心不是“更小的微内核”，而是：

Kernel protects and multiplexes physical resources; applications choose abstractions/policies

也就是把传统 OS 为所有程序统一规定的高层抽象下放，让 library OS / application 更直接决定资源如何使用。

408 做题时抓住：exokernel 倾向于把资源保护留在内核，把更高层的资源管理和抽象选择交给应用/库。

结构题第一动作

不要一上来背“微内核可靠、宏内核快”。先画 privilege boundary：

哪些服务在内核？哪些在用户态？跨边界靠什么？

然后性能、隔离、可扩展性的结论自然出来。

7 第七章：系统引导——OS 自己也必须先被加载和获得控制权

7.1 Bootstrap Paradox：还没有 OS 时，谁来加载 OS？

OS 能装入应用程序，但开机时 OS 尚未运行。于是必须存在一条比 OS 更早的启动链：

Power On $\rightarrow$ Firmware $\rightarrow$ Boot Manager / Bootloader $\rightarrow$ Kernel $\rightarrow$ Initial User-space System

这就是 bootstrap：使用一个更小、更早期运行的环境，把更完整的系统加载起来。

7.2 Firmware：先把平台带到可加载下一阶段的状态

Firmware 负责平台早期初始化与启动策略。现代 UEFI 模型中，firmware 内的 Boot Manager 会按照 boot configuration 选择并加载 UEFI driver/application，包括 OS boot loader，并把控制传给下一阶段。

因此不要把 BIOS/UEFI 与 bootloader 混成同一个对象：

Firmware $\neq$ OS Bootloader $\neq$ Kernel

7.3 Bootloader：找到内核、准备参数、移交控制

不同平台和 OS 的细节差异很大，但抽象职责稳定：

1. 找到可启动的 kernel image；
2. 根据协议把 kernel/必要数据放到合适位置；
3. 准备启动参数和机器状态；
4. 跳转到 kernel entry，把控制权交给内核。

7.4 Kernel 启动之后还没有“完整用户环境”

Kernel 接管机器后仍需初始化核心子系统，并创建/启动第一个用户态管理实体，之后才逐步形成服务、登录环境和普通应用。

因此开机过程不是：

Power $\rightarrow$ OS appears

而是一个控制权逐级交接和环境逐步扩充的过程。

引导题信号

遇到“ROM/firmware/bootloader/kernel 谁先谁后”时，只做一件事：追踪下一阶段代码是谁加载的、控制权交给了谁。

8 第八章：虚拟机——OS 虚拟化资源，不等于 VMM 虚拟整台机器

8.1 两种 virtualization 必须分层

OS 内部的资源虚拟化：

Hardware $\rightarrow$ Operating System $\rightarrow$ Processes / Address Spaces / Files

机器级虚拟化：

Hardware $\rightarrow$ VMM / Hypervisor $\rightarrow$ Virtual Machine $\rightarrow$ Guest OS $\rightarrow$ Applications

前者给应用程序提供 OS abstraction；后者给另一个操作系统提供一台近似完整的 virtual hardware machine。

8.2 VMM 的母问题：多个 OS 怎样安全共享同一台物理机器？

VMM 必须虚拟化/仲裁：

- CPU execution；
- memory；
- interrupt / timer；
- virtual devices / I/O；
- isolation between VMs。

它与普通进程隔离相似，但边界更低：guest OS 本身认为自己在控制机器。

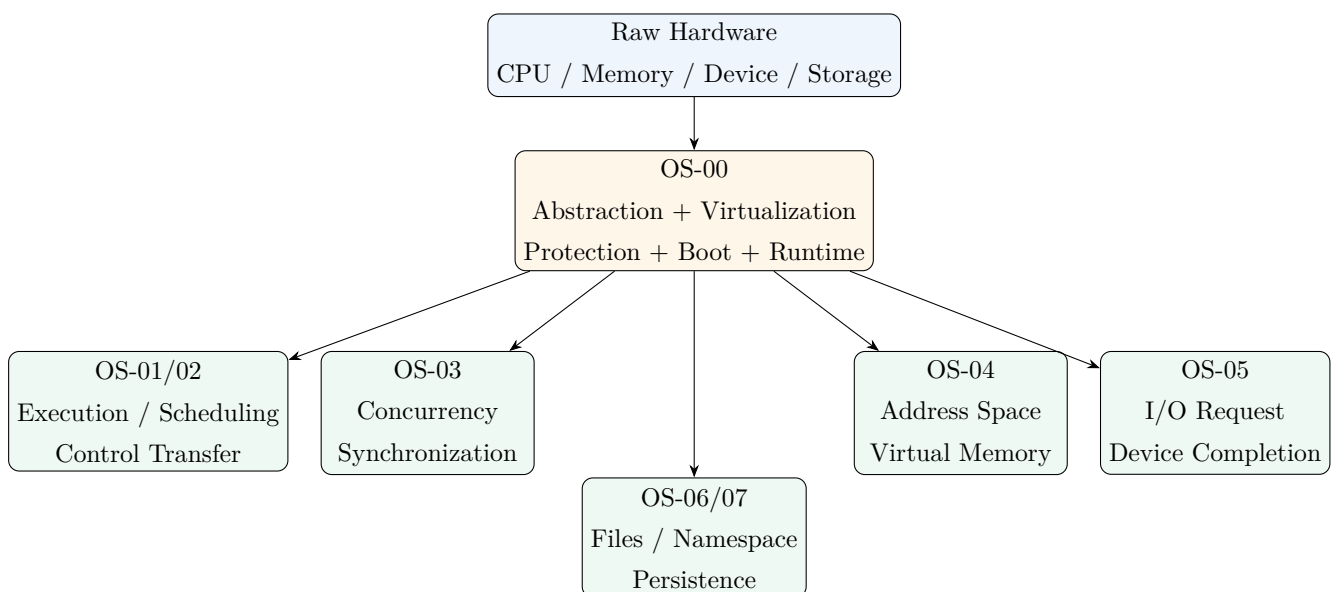
8.3 Type 1 / Type 2 只作为部署结构理解

经典教材常区分：

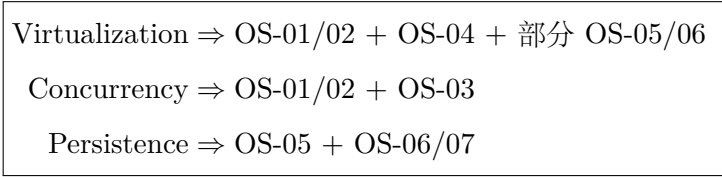
- Type 1: hypervisor 更直接运行在硬件之上；
- Type 2: VMM 作为 host OS 上的软件运行。

不要把分类当成绝对实现定律；现代虚拟化系统可能混合 host kernel module、user-space VMM、hardware virtualization support 等多层组件。408 题按教材抽象判断即可。

9 第九章：一张总图把 OS-00 挂到后续所有 Topic



OSTEP 三条主线也可挂回这个总图：



10 第十章：OS-00 做题控制器——先定位层，再调用 Owner

10.1 七问协议

看到操作系统基础题，按顺序问：

- 1. 题目现在谈的是物理资源、OS 抽象，还是程序可见对象？
- 2. 它在问 abstraction、virtualization、protection 还是 policy？
- 3. 并发/共享/虚拟/异步中，题面真正描述的是哪一性质？
- 4. 当前对象是 program、object file、executable、process image 还是 process？
- 5. 题目在追踪数据/符号绑定，还是追踪 CPU 控制权？
- 6. 题目在比较 OS 结构时，privilege boundary 画在哪里？
- 7. 若已经进入进程、同步、页表、I/O、文件内部机制，应该切到哪个 Canonical Owner？

10.2 高频概念区分清单

必须区分	判断抓手
Abstraction ≠ Virtualization	接口简化 vs 资源逻辑化/复用
Concurrency ≠ Parallelism	时间段内共同推进 vs 同一时刻同时执行
Mechanism ≠ Policy	如何做到 vs 选择谁/选择什么
Program ≠ Process	静态描述 vs 动态执行实体
Executable ≠ Process Image	文件静态表示 vs 某次执行的运行时表示
Loading ≠ All Pages Resident	建立运行映像/映射不要求所有页立即进入物理内存
Task State ≠ CPU Mode	Ready/Blocked 等 vs User/Kernel privilege
Monolithic ≠ Non-modular	内核边界位置 vs 软件组织方式
Firmware ≠ Bootloader ≠ Kernel	早期平台环境 vs OS 装载阶段 vs OS 核心
OS Virtualization ≠ Machine Virtualization	给应用虚拟资源 vs 给 guest OS 虚拟机器

10.3 忘掉名词时怎样重新推出来？

OS-00 最重要的不是背十几个定义，而是保留一条生成链：

Hardware is finite / complex / shared $\Rightarrow$ Need abstraction and management
 Programs must coexist $\Rightarrow$ Need virtualization + protection + policy
 Activities interleave $\Rightarrow$ Concurrency + asynchrony problems
 Programs start from static files $\Rightarrow$ Link + Load + Runtime Image
 OS itself is software $\Rightarrow$ Need bootstrapping
 Kernel is trusted $\Rightarrow$ Need a privilege boundary and an architecture choice
 Whole OS may itself be virtualized $\Rightarrow$ Need VMM / Hypervisor

本册最终心智模型

看到“操作系统”时，不再只想到“管理软硬件资源”。应该自动看到四层：

Physical Resource $\rightarrow$ Protected OS Mechanism $\rightarrow$ Managed Abstraction $\rightarrow$ Program-visible Environment

然后继续追问：

谁共享？谁隔离？谁决策？什么时候绑定？控制权现在在哪一层？

只要题目进入某个局部机制，就切换到对应 Topic；OS-00 的职责是**把问题送到正确的 Owner**，而不是把所有 OS 知识都重讲一遍。

参考与工程边界来源

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Version 1.10: Introduction、Virtualization、Concurrency、Persistence、Virtual Machines。
- System V Application Binary Interface, ELF: *Program Loading and Dynamic Linking*。用于 executable/shared object、process image 与 dynamic linker 的工程边界。
- GNU Binutils, 1d documentation: 链接器的 symbol resolution / relocation 基础语义。
- UEFI Specification 2.10, Chapter 3 *Boot Manager*: firmware boot manager 与 OS loader 的启动边界。
- Linux Kernel Documentation, x86 boot protocol: bootloader 与 kernel entry 的工程实例。
- seL4 Documentation / FAQ: microkernel minimal privileged core、user-level services 与 capability-based protection 的工程边界。
- MIT Exokernel research: separating protection from management、application-level resource management 的经典思想。
- 2026 计算机学科专业基础综合（408）考试大纲：操作系统基础、程序运行环境、OS 结构、引导与虚拟机范围。